

Build Your Own Plugin

Turn a proven AI capabilities into **versioned infrastructure** your whole team can install with one command

KEEP THIS PAGE ▶ It doubles as a prompt. When you're ready to build, **drop this PDF into Claude Code** and fill in the brief in 04. This page contains everything Claude needs to scaffold, test, and package your plugin.

01 WHAT A PLUGIN IS

A Claude Code plugin bundles components like skills, agents, hooks and MCP servers, into a single shareable **folder that anyone with access can install**. Plugins are powerful because they enable teams to turn know-how into shareable infrastructure. Knowledge, workflows, methodologies and guardrails can be installed like a package, versioned like code, and governed centrally, so teams can adopt AI with more predictability and standardization.

02 WHEN TO BUILD ONE (AND WHEN NOT TO)

Build a plugin when a workflow must **reach other people and stay consistent** as it spreads. Reach for something lighter when the scope is narrower:

A capability only you need → **a personal skill** `~/.claude/skills`

A rule true for a single repo → **that repo's** `CLAUDE.md`

External services with nothing to standardize → **plain MCP servers**

RULE OF THUMB: If several engineers keep solving the same problem through different approaches, it's a great candidate to be a plugin

03 CORE BUILDING BLOCKS

PRIMITIVE	WHAT IT DOES	WHAT YOU BRING: CLAUDE CAN'T INFER THIS
Skill <code>skills/<name>/SKILL.md</code>	A packaged procedure plus its supporting files. Runs when you type <code>/skill-name</code> or automatically when the task matches its description.	The steps , inputs and outputs, and a precise "use when..." description.
Subagent <code>agents/<name>.md</code>	An isolated worker with its own context window, prompt, and scoped tools.	Which tools it may touch, and when Claude should delegate to it.
Hook <code>hooks/hooks.json</code>	A script bound to a lifecycle event (e.g. <code>PostToolUse</code>). Fires deterministically. No model judgment.	The event , the matcher, and the exact command to run.
MCP server <code>.mcp.json</code>	A live connection to an outside system, exposed to Claude as callable tools.	The system , its URL or command, and the env vars or keys it needs.

* Legacy `commands/` still loads, but new plugins put commands in `skills/`. Additional less often used primitives include: LSP servers, Monitors, Output Styles and Themes

04 BUILD IT WITH CLAUDE CODE

Claude Code writes the plugin files; **your job is to describe the outcome well**. State the goal, give context Claude can't infer, make it interview you, and hold it to standards. Copy and fill the `[brackets]`:

```
Windows PowerShell
I'm an engineer on [TEAM] at Capital One. Build a Claude Code plugin that [OUTCOME - the workflow to standardize, in one sentence].

Context:
• Users & scale - [who runs it, and how many]
• Where it runs - [repos, languages, CI, systems it touches]
• Inputs & triggers - [what starts it; what it reads and writes]
• Constraints - [security, data handling, approvals, credentials]

How to work:
1. Interview me first. Ask only what you can't infer above; call out any assumptions that aren't obvious.
2. Propose a plan. which primitives (skill / subagent / hook / MCP server, etc.) and why, plus the folder layout. Wait for my approval.
3. Build one primitive at a time. After each, give me the exact local test and the expected result. No scope I didn't ask for.

Apply best-practices at every step:
Focused, reusable, predictable, easy to install and well documented

When it passes, package it: .claude-plugin/plugin.json (name, description, version) plus a README, and give me the exact commands to test, validate, and publish it.
```

05 TEST → SHIP → HAND OFF

- TEST LOCALLY**
`claude --plugin-dir ./my-plugin`
Loads your work-in-progress without installing anything.
- ITERATE**
`/reload-plugins`
Picks up your edits mid-session; no restart.
- VALIDATE**
`claude plugin validate ./my-plugin --strict`
Catches manifest and frontmatter errors before a teammate does.
- PUBLISH**
`git push → .claude-plugin/marketplace.json`
A private repo keeps it internal. Claude writes the marketplace file too.
- TEAM INSTALLS**
`/plugin marketplace add <org>/<repo>`
`/plugin install <name>@<marketplace>`
One command per teammate; version bumps roll out to everyone.

HANDOFF TEST

One teammate installs it fresh and runs it with **zero help from you**. Every question they ask is a README gap; fix the README, not the teammate.